

Diagnosing Illinois: An Econometric and Machine Learning View of County Unemployment

Mingyang Li

This part is the foundation of the entire analysis. Real-world data is often scattered across different files (such as unemployment rates, education levels, poverty rates, and racial distribution). We need to merge them into a single comprehensive dataset.

The key technique here is fixing the FIPS codes. Since the state code for Illinois is 17, some raw datasets might have lost this prefix (for example, turning 17031 into 31). We forcefully pad and fix them in this step so that subsequent table merges and spatial mapping won't fail due to missing data matches.

```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from sklearn.linear_model import LinearRegression
import urllib.request
import json
import warnings

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')

# -----
# Part 1: Data Cleaning & Merging
# -----

# Helper function to clean and fix FIPS codes
def clean_fips(series):
    # Strip whitespace and remove trailing '.0'
    s = series.astype(str).str.strip().str.replace(r'\.0$', '', regex=True)
```

```

# Append Illinois state prefix '17' if missing (length <= 3)
return s.apply(lambda x: '17' + x.zfill(3) if len(x) <= 3 else x.zfill(5))

# 1. Load and clean main Unemployment data
df_main = pd.read_csv("raw-data/UnemploymentReport.csv")
df_main.columns = df_main.columns.str.strip()
df_main["FIPS"] = clean_fips(df_main["FIPS"])
df_main["Merge_Name"] = df_main["Name"].str.replace(" County, IL", "", regex=False).str.lower

# 2. Load and clean Education data (Bachelor's Degree %)
df_edu = pd.read_csv("raw-data/bachelor.csv")
fips_col = [col for col in df_edu.columns if 'FIPS' in col.upper()][0]
df_edu = df_edu[[fips_col, "2019-2023"]].rename(columns={fips_col: "FIPS", "2019-2023": "Bach_Pct"})
df_edu["Bach_Pct"] = pd.to_numeric(df_edu["Bach_Pct"].astype(str).str.replace('%', ''), errors='coerce')
df_edu["FIPS"] = clean_fips(df_edu["FIPS"])

# 3. Load and clean Poverty data
df_pov_raw = pd.read_csv("raw-data/PovertyReport.csv", skiprows=2)
df_pov = df_pov_raw.iloc[:, [0, 5]].copy()
df_pov.columns = ["FIPS", "Poverty_Pct"]
df_pov["FIPS"] = clean_fips(df_pov["FIPS"])
df_pov["Poverty_Pct"] = pd.to_numeric(df_pov["Poverty_Pct"], errors='coerce')

# 4. Load, transpose, and clean Census Race data (Black Population %)
df_race_raw = pd.read_csv("raw-data/DECENNIALDHC2020.P9-2026-03-04T220749.csv")
df_race_t = df_race_raw.transpose()
df_race_t.columns = df_race_t.iloc[0]
df_race_t = df_race_t.drop(df_race_t.index[0])
total_col = [c for c in df_race_t.columns if "Total:" in str(c)][0]
black_col = [c for c in df_race_t.columns if "Black or African American alone" in str(c)][0]

total_pop = pd.to_numeric(df_race_t[total_col].astype(str).str.replace(',', ''), errors='coerce')
black_pop = pd.to_numeric(df_race_t[black_col].astype(str).str.replace(',', ''), errors='coerce')
df_race_t["Black_Pct"] = (black_pop / total_pop) * 100
df_race_t["Merge_Name"] = df_race_t.index.str.replace(" County, Illinois", "", regex=False).str.lower

# 5. Execute multiple inner joins to combine all datasets
m1 = pd.merge(df_main, df_edu, on="FIPS", how="inner")
m2 = pd.merge(m1, df_pov, on="FIPS", how="inner")
df = pd.merge(m2, df_race_t[["Merge_Name", "Black_Pct"]], on="Merge_Name", how="inner")

# Convert target columns to numeric and drop any rows with missing values

```

```
cols_to_check = ["2023", "2020", "Bach_Pct", "Poverty_Pct", "Black_Pct"]
for c in cols_to_check:
    df[c] = pd.to_numeric(df[c], errors='coerce')
df = df.dropna(subset=cols_to_check)

print(f>Data Cleaning Complete. Total counties retained: {len(df)}")
```

Data Cleaning Complete. Total counties retained: 102

I use the 2020 pandemic unemployment rate, bachelor's degree percentage, poverty rate, and Black population percentage as input features (X) to predict the actual 2023 unemployment rate (y). The most crucial metric we calculate here is the Residual (Predicted Value - Actual Value). If the residual is exceptionally large, it means the county's unemployment rate deviates from the standard socioeconomic pattern, indicating potential unique local industries or policies that require deeper investigation.

```
# -----
# Part 2: Machine Learning Prediction & Residuals
# -----

# Define independent variables (X) and dependent variable (y)
X = df[["2020", "Bach_Pct", "Poverty_Pct", "Black_Pct"]]
y = df["2023"]

# Initialize and fit the Multiple Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Generate AI predictions based on the 4 socioeconomic factors
df["Predicted_2023"] = model.predict(X)

# Calculate Residuals (Predicted - Actual)
# Force as float to prevent Plotly continuous color scale errors
df["Residual"] = (df["Predicted_2023"] - df["2023"]).astype(float)

print("Machine Learning modeling complete. Predictions and Residuals added.")
```

Machine Learning modeling complete. Predictions and Residuals added.

Part 1: Data Cleaning & Merging

Explanation: This part is the foundation of the entire analysis. Real-world data is often scattered across different files (such as unemployment rates, education levels, poverty rates, and racial distribution). We need to merge them into a single comprehensive dataset.

The key technique here is fixing the FIPS codes. Since the state code for Illinois is 17, some raw datasets might have lost this prefix (for example, turning 17031 into 31). We forcefully pad and fix them in this step so that subsequent table merges and spatial mapping won't fail due to missing data matches.

```
Python import pandas as pd import numpy as np import plotly.express as px import
plotly.graph_objects as go from sklearn.linear_model import LinearRegression import
urllib.request import json import warnings
```

Suppress warnings for cleaner output

```
warnings.filterwarnings('ignore')
```

Part 1: Data Cleaning & Merging

Helper function to clean and fix FIPS codes

```
def clean_fips(series): # Strip whitespace and remove trailing '0's = series.astype(str).str.strip().str.re-
place(r'0$', '', regex=True) # Append Illinois state prefix '17' if missing (length <= 3) return
s.apply(lambda x: '17' + x.zfill(3) if len(x) <= 3 else x.zfill(5))
```

1. Load and clean main Unemployment data

```
df_main = pd.read_csv("UnemploymentReport.csv") df_main.columns = df_main.columns.str.strip()
df_main["FIPS"] = clean_fips(df_main["FIPS"]) df_main["Merge_Name"] = df_main["Name"].str.re-
place(" County, IL", "", regex=False).str.lower().str.replace(" ", "").str.replace(".", "",
regex=False)
```

2. Load and clean Education data (Bachelor's Degree %)

```
df_edu = pd.read_csv("bachelor.csv") fips_col = [col for col in df_edu.columns if 'FIPS'
in col.upper()][0] df_edu = df_edu[[fips_col, "2019-2023"]].rename(columns={fips_col: "FIPS",
"2019-2023": "Bach_Pct"}) df_edu["Bach_Pct"] = pd.to_numeric(df_edu["Bach_Pct"].astype(str).str.re-
place('%', ''), errors='coerce') df_edu["FIPS"] = clean_fips(df_edu["FIPS"])
```

3. Load and clean Poverty data

```
df_pov_raw = pd.read_csv("PovertyReport.csv", skiprows=2) df_pov = df_pov_raw.iloc[:, [0,
5]].copy() df_pov.columns = ["FIPS", "Poverty_Pct"] df_pov['FIPS'] = clean_fips(df_pov['FIPS'])
df_pov['Poverty_Pct'] = pd.to_numeric(df_pov['Poverty_Pct'], errors='coerce')
```

4. Load, transpose, and clean Census Race data (Black Population %)

```
df_race_raw = pd.read_csv("DECENNIALDHC2020.P9-2026-03-04T220749.csv") df_race_t
= df_race_raw.transpose() df_race_t.columns = df_race_t.iloc[0] df_race_t =
df_race_t.drop(df_race_t.index[0]) total_col = [c for c in df_race_t.columns if "To-
tal:" in str(c)][0] black_col = [c for c in df_race_t.columns if "Black or African American
alone" in str(c)][0]
total_pop = pd.to_numeric(df_race_t[total_col].astype(str).str.replace(',', ''), er-
rors='coerce') black_pop = pd.to_numeric(df_race_t[black_col].astype(str).str.re-
place(',', ''), errors='coerce') df_race_t["Black_Pct"] = (black_pop / total_pop) *
100 df_race_t["Merge_Name"] = df_race_t.index.str.replace(" County, Illinois", "",
regex=False).str.lower().str.replace(" ", "").str.replace(".", "", regex=False)
```

5. Execute multiple inner joins to combine all datasets

```
m1 = pd.merge(df_main, df_edu, on="FIPS", how="inner") m2 = pd.merge(m1, df_pov,
on="FIPS", how="inner") df = pd.merge(m2, df_race_t[["Merge_Name", "Black_Pct"]],
on="Merge_Name", how="inner")
```

Convert target columns to numeric and drop any rows with missing values

```
cols_to_check = ["2023", "2020", "Bach_Pct", "Poverty_Pct", "Black_Pct"]
for c in cols_to_check: df[c] = pd.to_numeric(df[c], errors='coerce')
df = df.dropna(subset=cols_to_check)
```

```
print(f"Data Cleaning Complete. Total counties retained: {len(df)}")
```

Part 2: Machine Learning Prediction & Residuals

Explanation: We are not just looking at historical data; we are trying to uncover hidden patterns. Here, we introduce the Multiple Linear Regression model from scikit-learn Multiple Linear Regression, AI generated Shutterstock Explore

We use the 2020 pandemic unemployment rate, bachelor's degree percentage, poverty rate, and Black population percentage as input features (X) to predict the actual 2023 unemployment rate (y). The most crucial metric we calculate here is the Residual (Predicted Value - Actual Value). If the residual is exceptionally large, it means the county's unemployment rate deviates from the standard socioeconomic pattern, indicating potential unique local industries or policies that require deeper investigation.

```
Python # _____ # Part 2: Machine Learning Prediction
& Residuals # _____
```

Define independent variables (X) and dependent variable (y)

```
X = df[["2020", "Bach_Pct", "Poverty_Pct", "Black_Pct"]]
y = df["2023"]
```

Initialize and fit the Multiple Linear Regression model

```
model = LinearRegression()
model.fit(X, y)
```

Generate AI predictions based on the 4 socioeconomic factors

```
df["Predicted_2023"] = model.predict(X)
```

Calculate Residuals (Predicted - Actual)

Force as float to prevent Plotly continuous color scale errors

```
df["Residual"] = (df["Predicted_2023"] - df["2023"]).astype(float)
```

```
print("Machine Learning modeling complete. Predictions and Residuals added.") Part 3:  
Advanced Visualizations
```

Explanation: Now that our data is fully prepped, the next step is storytelling through visual analytics. We use Plotly to create interactive, hoverable charts:

Boxplot: Displays the overall distribution of the 2023 unemployment rates and helps identify extreme outliers.

Multivariate Quadrant Scatter: The X-axis represents poverty, the Y-axis is unemployment, the bubble size indicates education levels, and the color scale reflects our calculated "residual". This single chart visualizes 4 dimensions of data!

Predicted vs Actual: Draws a diagonal reference line. The closer the data points are to this line, the more accurate our regression model is.

Correlation Heatmap: Visually demonstrates the strength of relationships between all socioeconomic variables.

Leaderboard: A bar chart that intuitively highlights the top 20 counties suffering from the highest unemployment rates in 2023.

```
# -----  
# Part 3: Advanced Visualizations  
# -----  
plotly_theme = "plotly_dark" # Use dark theme for dashboard aesthetics  
  
# 1. Boxplot with all data points  
fig_box = px.box(df, y="2023", points="all", template=plotly_theme,  
                 title="Distribution of 2023 Unemployment Rates",  
                 labels={"2023": "2023 Unemployment (%)"})  
fig_box.update_traces(marker=dict(color='#90CAF9', size=6), boxmean=True)  
fig_box.show()  
  
# 2. Multivariate Quadrant Scatter Plot  
fig_scatter = px.scatter(  
    df,  
    x="Poverty_Pct",  
    y="2023",
```

```

        color="Residual",
        size="Bach_Pct",
        hover_name="Name",
        color_continuous_scale="RdBu_r",
        template=plotly_theme,
        title="Multivariate Quadrant (Poverty vs Unemployment)",
        labels={"Poverty_Pct": "Poverty Rate (%)", "2023": "2023 Unemployment (%)"}
    )
fig_scatter.show()

# 3. Predicted vs Actual Model Diagnostics
fig_pred = px.scatter(df, x="2023", y="Predicted_2023", hover_name="Name", trendline="ols",
                    template=plotly_theme, title="Predicted vs Actual Unemployment (2023)",
                    labels={"2023": "Actual 2023 (%)", "Predicted_2023": "Predicted 2023 (%)"}

# Add Y=X baseline for perfect prediction reference
min_val = min(df["2023"].min(), df["Predicted_2023"].min())
max_val = max(df["2023"].max(), df["Predicted_2023"].max())
fig_pred.add_shape(type="line", x0=min_val, y0=min_val, x1=max_val, y1=max_val, line=dict(color="black"))
fig_pred.show()

# 4. Correlation Matrix Heatmap
corr_cols = ["2020", "Bach_Pct", "Poverty_Pct", "Black_Pct", "2023", "Predicted_2023", "Residual"]
corr_matrix = df[corr_cols].corr()
fig_heat = px.imshow(corr_matrix, text_auto=".2f", aspect="auto", color_continuous_scale="RdBu_r",
                    template=plotly_theme, title="Pearson Correlation Heatmap")
fig_heat.show()

# 5. Leaderboard Bar Chart (Top 20 Counties)
df_sorted = df.sort_values(by="2023", ascending=False).head(20)
fig_leader = px.bar(df_sorted, x="Name", y="2023", template=plotly_theme,
                    title="Leaderboard (Top 20 Highest Unemployment Counties)",
                    color_discrete_sequence=["#90CAF9"])
fig_leader.show()

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Part 4: Spatial Map Intelligence

Explanation: The final step is projecting our findings onto real-world geography .

Instead of requiring you to download a local .geojson or .shp shapefile, this code dynamically fetches a standardized US counties boundary file directly from Github via Python. Using our successfully repaired FIPS codes, we map out the “Prediction Residuals” using a choropleth map. Blue areas indicate counties performing better than expected, while red areas are worse off than the model predicted.

We generate two distinct views: a statewide overview of Illinois and a zoomed-in perspective of the Chicago metropolitan area.

```
# -----
# Part 4: Interactive Spatial Map Intelligence
# -----

print("Fetching US Counties GeoJSON from web...")
# Dynamically pull spatial geometry data via URL
url = "https://raw.githubusercontent.com/plotly/datasets/master/geojson-counties-fips.json"
with urllib.request.urlopen(url) as response:
    counties_geojson = json.load(response)

# 1. Statewide Choropleth Map showing Residuals
fig_map_state = px.choropleth_mapbox(
    df, geojson=counties_geojson, locations='FIPS', color='Residual',
    color_continuous_scale="RdBu_r", mapbox_style="carto-darkmatter",
    zoom=5, center={"lat": 40.0, "lon": -89.0}, opacity=0.8,
    hover_name="Name", template=plotly_theme,
    title="Spatial Intelligence: Residual Diagnostics (Illinois Statewide)"
)
fig_map_state.show()

# 2. Chicago Metropolitan Area Zoom-in Map
fig_map_chi = px.choropleth_mapbox(
    df, geojson=counties_geojson, locations='FIPS', color='Residual',
    color_continuous_scale="RdBu_r", mapbox_style="carto-darkmatter",
```

```

    zoom=7.5, center={"lat": 41.8, "lon": -87.8}, opacity=0.8,
    hover_name="Name", template=plotly_theme,
    title="Spatial Intelligence: Residual Diagnostics (Chicago Zoom)"
)
fig_map_chi.show()

```

Fetching US Counties GeoJSON from web...

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Part 5: OLS Statistical Inference & Residual Diagnostics

This section upgrades the analysis from a simple sklearn prediction to a rigorous econometric model using `statsmodels`. Unlike sklearn, `statsmodels` produces a full OLS summary table with coefficients, standard errors, t-statistics, and p-values — the standard output expected in academic and policy contexts. We also generate a Q-Q plot to visually test whether the residuals follow a normal distribution, which is a key assumption of linear regression. If the points fall along the diagonal, the assumption holds.

```

# -----
# Part 5: OLS Statistical Inference & Q-Q Plot
# -----
import statsmodels.api as sm
from scipy import stats

# Add constant for intercept term
X_ols = sm.add_constant(df[["2020", "Bach_Pct", "Poverty_Pct", "Black_Pct"]])
y_ols = df["2023"]

# Fit OLS model
ols_model = sm.OLS(y_ols, X_ols).fit()

# Print full regression summary (coefficients, p-values, R2, F-stat)
print(ols_model.summary())

# --- Q-Q Plot of Residuals ---
residuals = ols_model.resid
(osm, osr), (slope, intercept, r) = stats.probplot(residuals, dist="norm")

```

```

fig_qq = go.Figure()
fig_qq.add_trace(go.Scatter(
    x=osm, y=osr,
    mode='markers',
    marker=dict(color='#90CAF9', size=6),
    name='Residuals'
))
fig_qq.add_trace(go.Scatter(
    x=osm, y=slope * np.array(osm) + intercept,
    mode='lines',
    line=dict(color='red', dash='dash'),
    name='Normal Reference Line'
))
fig_qq.update_layout(
    title="Q-Q Plot: Are Residuals Normally Distributed?",
    xaxis_title="Theoretical Quantiles",
    yaxis_title="Sample Quantiles",
    template=plotly_theme
)
fig_qq.show()

```

OLS Regression Results

```

=====
Dep. Variable:          2023      R-squared:                0.620
Model:                  OLS      Adj. R-squared:         0.605
Method:                 Least Squares  F-statistic:           39.64
Date:                  Sun, 08 Mar 2026  Prob (F-statistic):    1.23e-
19
Time:                  17:18:21   Log-Likelihood:        -
78.497
No. Observations:      102      AIC:                  167.0
Df Residuals:          97      BIC:                  180.1
Df Model:              4
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	1.9136	0.433	4.421	0.000	1.054	2.773
2020	0.3871	0.041	9.391	0.000	0.305	0.469
Bach_Pct	-0.0226	0.008	-2.812	0.006	-0.038	-
0.007						
Poverty_Pct	0.0079	0.020	0.395	0.693	-0.032	0.047

Black_Pct	0.0095	0.011	0.899	0.371	-0.011	0.030
-----------	--------	-------	-------	-------	--------	-------

Omnibus:	4.345	Durbin-Watson:	1.245
Prob(Omnibus):	0.114	Jarque-Bera (JB):	4.851
Skew:	-0.186	Prob(JB):	0.0884
Kurtosis:	4.001	Cond. No.	236.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Unable to display output for mime type(s): text/html

Part 6: Time Dimension — Recovery Analysis (2020 → 2023)

The pandemic year 2020 saw a dramatic spike in unemployment across all counties. By comparing 2020 and 2023 figures, we can compute a “Recovery Score” — how much each county has bounced back. Counties with the smallest recovery (or even negative recovery) may still be structurally distressed, while fast-recovering counties may reveal resilient local economies. This temporal lens adds a dimension missing from purely cross-sectional analysis.

```
# -----
# Part 6: Temporal Recovery Analysis (2020 → 2023)
# -----

# Compute Recovery Score: positive = improved, negative = worsened
df["Recovery"] = df["2020"] - df["2023"]

# 1. Scatter: 2020 vs 2023 with recovery color gradient
fig_recovery_scatter = px.scatter(
    df, x="2020", y="2023",
    color="Recovery",
    color_continuous_scale="RdYlGn",
    hover_name="Name",
    size=abs(df["Recovery"] + 0.01),
    template=plotly_theme,
    title="Pandemic Recovery: 2020 vs 2023 Unemployment by County",
    labels={"2020": "2020 Unemployment (%)", "2023": "2023 Unemployment (%)"}
)

# Add Y=X diagonal: counties ON the line had zero recovery
min_v = min(df["2020"].min(), df["2023"].min())
max_v = max(df["2020"].max(), df["2023"].max())
```

```

fig_recovery_scatter.add_shape(
    type="line", x0=min_v, y0=min_v, x1=max_v, y1=max_v,
    line=dict(color="gray", dash="dot")
)
fig_recovery_scatter.add_annotation(
    x=max_v * 0.85, y=max_v * 0.92,
    text="+ No Recovery Line", showarrow=False,
    font=dict(color="gray", size=11)
)
fig_recovery_scatter.show()

# 2. Bar chart: Top 10 fastest recovering vs 10 slowest
df_rec_sorted = df.sort_values("Recovery", ascending=False)
top10 = df_rec_sorted.head(10).copy()
bot10 = df_rec_sorted.tail(10).copy()
df_extremes = pd.concat([top10, bot10])
df_extremes["Group"] = ["Fast Recovery"] * 10 + ["Slow/No Recovery"] * 10

fig_recovery_bar = px.bar(
    df_extremes, x="Name", y="Recovery",
    color="Group",
    color_discrete_map={"Fast Recovery": "#66BB6A", "Slow/No Recovery": "#EF5350"},
    template=plotly_theme,
    title="Top 10 Fastest vs Slowest Recovering Counties (2020-2023)",
    labels={"Recovery": "Unemployment Drop (pp)", "Name": "County"}
)
fig_recovery_bar.update_layout(xaxis_tickangle=-45)
fig_recovery_bar.show()

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Part 7: Single-Variable Deep Dive & Quantile Analysis

Rather than collapsing all relationships into a single heatmap correlation number, this section examines each feature's relationship with unemployment individually. The 2×2 subplot grid gives a clear picture of linearity (or lack thereof) for each predictor. We then perform a quantile split on Poverty Rate — dividing counties into Low, Medium, and High poverty terciles — and compare average unemployment across groups. This bridges descriptive statistics with causal intuition.

```

# -----
# Part 7: Single-Variable Scatter Subplots & Quantile Analysis
# -----
from plotly.subplots import make_subplots

features = ["Poverty_Pct", "Black_Pct", "Bach_Pct", "2020"]
feature_labels = ["Poverty Rate (%)", "Black Population (%)", "Bachelor's Degree (%)", "2020"]

fig_sub = make_subplots(
    rows=2, cols=2,
    subplot_titles=[f"{fl} vs 2023 Unemployment" for fl in feature_labels]
)

colors = ["#EF5350", "#AB47BC", "#42A5F5", "#26A69A"]

for i, (feat, label, color) in enumerate(zip(features, feature_labels, colors)):
    row, col = divmod(i, 2)
    # Scatter points
    fig_sub.add_trace(
        go.Scatter(
            x=df[feat], y=df["2023"],
            mode="markers",
            marker=dict(color=color, size=7, opacity=0.75),
            text=df["Name"],
            hovertemplate="%{text}<br>" + label + ": %{x:.1f}<br>Unemployment: %{y:.1f}%",
            name=label,
            showlegend=False
        ),
        row=row+1, col=col+1
    )
    # OLS trendline manually
    m_c, b_c = np.polyfit(df[feat].dropna(), df.loc[df[feat].notna(), "2023"], 1)
    x_line = np.linspace(df[feat].min(), df[feat].max(), 50)
    fig_sub.add_trace(
        go.Scatter(
            x=x_line, y=m_c * x_line + b_c,
            mode="lines",
            line=dict(color="white", dash="dash", width=1.5),
            showlegend=False
        ),
        row=row+1, col=col+1
    )

```

```

fig_sub.update_layout(
    height=700, template=plotly_theme,
    title_text="Single-Variable Relationships with 2023 Unemployment"
)
fig_sub.show()

# --- Quantile Split on Poverty Rate ---
df["Poverty_Tier"] = pd.qcut(df["Poverty_Pct"], q=3, labels=["Low Poverty", "Medium Poverty", "High Poverty"])
tier_stats = df.groupby("Poverty_Tier", observed=True)["2023"].agg(["mean", "median", "std"])
tier_stats.columns = ["Poverty Tier", "Mean Unemployment", "Median Unemployment", "Std Dev"]

fig_quantile = px.bar(
    tier_stats, x="Poverty Tier",
    y=["Mean Unemployment", "Median Unemployment"],
    barmode="group",
    template=plotly_theme,
    title="Unemployment by Poverty Tercile (Quantile Analysis)",
    color_discrete_sequence=["#EF5350", "#90CAF9"]
)
fig_quantile.show()

print(tier_stats.to_string(index=False))

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Poverty Tier	Mean Unemployment	Median Unemployment	Std Dev
Low Poverty	4.317500	4.15	0.839685
Medium Poverty	4.387097	4.40	0.654595
High Poverty	5.187097	5.20	0.768003

Part 8: K-Means Clustering — Economic Typology Map

Instead of viewing each county in isolation, K-Means clustering groups counties into distinct economic archetypes based on all four features simultaneously. With $k=4$, we typically recover interpretable cluster profiles: e.g., Cluster A = high poverty / high unemployment (distressed rural), Cluster B = high education / low unemployment (suburban), etc. Mapping these clusters geographically reveals spatial concentrations of economic type — a finding much richer than any single-variable map.

```

# -----
# Part 8: K-Means Clustering & Economic Typology Map
# -----
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Features used for clustering
cluster_features = ["2023", "Poverty_Pct", "Bach_Pct", "Black_Pct", "2020"]
X_cluster = df[cluster_features].dropna()
idx = X_cluster.index

# Standardize before clustering
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cluster)

# Fit K-Means with k=4
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
df.loc[idx, "Cluster"] = kmeans.fit_predict(X_scaled).astype(str)

# Build interpretable cluster profile table
cluster_profile = df.groupby("Cluster")[cluster_features].mean().round(2)
cluster_profile.index = [f"Cluster {i}" for i in cluster_profile.index]
print("=== Cluster Economic Profiles ===")
print(cluster_profile.to_string())

# Assign human-readable labels based on unemployment rank
unemp_rank = cluster_profile["2023"].rank()
label_map = {}
for c in cluster_profile.index:
    u = cluster_profile.loc[c, "2023"]
    p = cluster_profile.loc[c, "Poverty_Pct"]
    b = cluster_profile.loc[c, "Bach_Pct"]
    if u == cluster_profile["2023"].max():
        label_map[c.replace("Cluster ", "")] = "High Unemployment / Distressed"
    elif u == cluster_profile["2023"].min():
        label_map[c.replace("Cluster ", "")] = "Low Unemployment / Affluent"
    elif b == cluster_profile["Bach_Pct"].max():
        label_map[c.replace("Cluster ", "")] = "Educated Suburban"
    else:
        label_map[c.replace("Cluster ", "")] = "Mid-tier / Mixed Economy"

df["Cluster_Label"] = df["Cluster"].map(label_map)

```



```
# Choropleth map of clusters
cluster_colors = {"High Unemployment / Distressed": "#EF5350",
                  "Low Unemployment / Affluent": "#66BB6A",
                  "Educated Suburban": "#42A5F5",
                  "Mid-tier / Mixed Economy": "#FFA726"}

fig_cluster_map = px.choropleth_mapbox(
    df.dropna(subset=["Cluster_Label"]),
    geojson=counties_geojson,
    locations="FIPS",
    color="Cluster_Label",
    color_discrete_map=cluster_colors,
    mapbox_style="carto-darkmatter",
    zoom=5, center={"lat": 40.0, "lon": -89.0},
    opacity=0.8,
    hover_name="Name",
    hover_data={"2023": True, "Poverty_Pct": True, "Bach_Pct": True},
    template=plotly_theme,
    title="K-Means Economic Typology Map (k=4)"
)
fig_cluster_map.show()
```

```
=== Cluster Economic Profiles ===
```

	2023	Poverty_Pct	Bach_Pct	Black_Pct	2020
Cluster 0	4.27	11.81	20.39	2.17	7.02
Cluster 1	4.11	8.81	37.47	5.82	7.61
Cluster 2	5.81	17.43	25.76	23.00	10.54
Cluster 3	5.26	16.19	19.22	5.31	9.20

Unable to display output for mime type(s): text/html

Part 9: Outlier County Deep Dive — Top 5 & Bottom 5 Residuals

The residual is the gap between what the model predicts and what actually happened. Counties with the largest positive residuals are performing *worse* than their demographics suggest — potential signs of structural collapse, brain drain, or lack of investment. Counties with the most negative residuals are *beating* expectations — possibly reflecting strong local policy, anchor institutions (universities, hospitals), or undercounted economic activity. This section surfaces and profiles these 10 outlier counties explicitly.

```

# -----
# Part 9: Outlier County Profile - Top 5 & Bottom 5 Residuals
# -----

# Sort by residual
df_sorted_res = df.sort_values("Residual", ascending=False)
top5_over = df_sorted_res.head(5) # Model under-predicts → worse than expected
bot5_under = df_sorted_res.tail(5) # Model over-predicts → better than expected

profile_cols = ["Name", "2023", "Predicted_2023", "Residual", "Poverty_Pct", "Bach_Pct", "BL

print("=== TOP 5: WORSE THAN EXPECTED (Positive Residual) ===")
print(top5_over[profile_cols].to_string(index=False))
print("\n=== BOTTOM 5: BETTER THAN EXPECTED (Negative Residual) ===")
print(bot5_under[profile_cols].to_string(index=False))

# Combined lollipop / dot-range chart
df_outliers = pd.concat([
    top5_over[profile_cols].assign(Group="Underperforming (Worse than Expected)"),
    bot5_under[profile_cols].assign(Group="Overperforming (Better than Expected)")
])

fig_outlier = px.scatter(
    df_outliers,
    x="Residual", y="Name",
    color="Group",
    color_discrete_map={
        "Underperforming (Worse than Expected)": "#EF5350",
        "Overperforming (Better than Expected)": "#66BB6A"
    },
    size=abs(df_outliers["Residual"]),
    hover_data={"2023": True, "Predicted_2023": True, "Poverty_Pct": True, "Bach_Pct": True},
    template=plotly_theme,
    title="Outlier Counties: Residual Deep Dive (Top 5 vs Bottom 5)"
)

# Add vertical zero line
fig_outlier.add_vline(x=0, line_dash="dot", line_color="gray")
fig_outlier.update_layout(xaxis_title="Residual (Predicted - Actual Unemployment %)", yaxis_t
fig_outlier.show()

# Radar / spider chart to compare average profiles of the two outlier groups

```

```

categories = ["2023", "Poverty_Pct", "Bach_Pct", "Black_Pct", "2020"]
cat_labels = ["Unemployment 2023", "Poverty %", "Bachelor's %", "Black Pop %", "Unemployment

over_avg = top5_over[categories].mean().tolist()
under_avg = bot5_under[categories].mean().tolist()

fig_radar = go.Figure()
for name, values, color in [
    ("Underperforming Counties", over_avg, "#EF5350"),
    ("Overperforming Counties", under_avg, "#66BB6A")
]:
    fig_radar.add_trace(go.Scatterpolar(
        r=values + [values[0]],
        theta=cat_labels + [cat_labels[0]],
        fill='toself',
        name=name,
        line=dict(color=color)
    ))

fig_radar.update_layout(
    polar=dict(radialaxis=dict(visible=True)),
    title="Radar Profile: Underperforming vs Overperforming Outlier Counties",
    template=plotly_theme
)
fig_radar.show()

```

=== TOP 5: WORSE THAN EXPECTED (Positive Residual) ===

	Name	2023	Predicted_2023	Residual	Poverty_Pct	Bach_Pct	Black_Pct	2020
	Wabash County, IL	3.4	5.361596	1.961596	14	18.7	0.466508	9.7
	Cook County, IL	4.4	5.387861	0.987861	13	41.9	22.473543	10.6
	Brown County, IL	2.4	3.350418	0.950418	14	15.1	16.575913	3.9
	Jefferson County, IL	4.7	5.636577	0.936577	15	18.4	7.498720	10.2
	Effingham County, IL	3.2	3.965071	0.765071	10	26.0	0.435560	6.6

=== BOTTOM 5: BETTER THAN EXPECTED (Negative Residual) ===

	Name	2023	Predicted_2023	Residual	Poverty_Pct	Bach_Pct	Black_Pct	2020
	Scott County, IL	5.1	4.094185	-1.005815	11	22.2	0.101031	6.7
	Pulaski County, IL	7.7	6.692533	-1.007467	22	11.9	28.153283	11.9
	Boone County, IL	7.0	5.872274	-1.127726	11	23.2	2.190915	11.3
	Stark County, IL	5.6	4.432614	-1.167386	11	19.3	0.314815	7.4
	Bureau County, IL	5.7	4.505798	-1.194202	11	21.4	0.779088	7.7

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html